In [1]:
```
!pip install -q pymupdf
!pip install -q scikit-image
!pip install -q jiwer
!pip install -q python-docx
!pip install -q numpy
!pip install -q Pillow
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.9/24.9 MB 73.5 MB/s eta 0:00:00:00:0100:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 50.2 MB/s eta 0:00:00a 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 253.0/253.0 kB 8.4 MB/s eta 0:00:00
```

In [2]:
```
import os
import re
import json
import time
import random
import warnings
import unicodedata
import numpy as np
import cv2
from PIL import Image
from pathlib import Path
from collections import Counter
from typing import List, Dict, Tuple, Optional
from io import BytesIO

import fitz  # pymupdf
from skimage.filters import threshold_sauvola
from jiwer import cer, wer

warnings.filterwarnings('ignore')

print('All imports successful')
```

```
All imports successful
```

In [3]:
```
# ============================================================
# GPT-4.1 SETUP
# ============================================================
```

```
!pip install -q openai

from openai import OpenAI
import base64
from io import BytesIO

OPENAI_API_KEY = 'openai-api-key'
client = OpenAI(api_key=OPENAI_API_KEY)

print('OpenAI client ready')
```

```
OpenAI client ready
```

In [4]:

```
# ============================================================
# CONFIGURATION
# ============================================================
import torch
BASE_DIR   = Path('/kaggle/input/datasets/indrasn0wal/humanai-handwriting-ocr/handwriting')
SCANS_DIR  = BASE_DIR / 'text scans'
TRANS_DIR  = BASE_DIR / 'transcription source'
OUTPUT_DIR = Path('/kaggle/working/outputs')
OUTPUT_DIR.mkdir(exist_ok=True)

SOURCES = {
    'source_1_1857': {
        'pdf': SCANS_DIR / 'PT3279_146_342 _ 1857.pdf',
        'gt_transcription': TRANS_DIR / 'PT3279.146.342 _ 1857_transcription.docx',
        'language': 'Spanish',
        'century': '17th'
    },
    'source_2_1744': {
        'pdf': SCANS_DIR / 'AHPG-GPAH 1_1716_A.35 _ 1744.pdf',
        'gt_transcription': TRANS_DIR / 'AHPG-GPAH 1.1716_A.35 _ 1744_transcription.docx',
        'language': 'Spanish',
        'century': '17th'
    },
    'source_3_pleito': {
        'pdf': SCANS_DIR / 'Pleito entre el Marque_s de Viana.pdf',
        'gt_transcription': TRANS_DIR / 'Pleito entre el Marque_s de Viana_transcription.docx',
        'language': 'Spanish',
        'century': '17th'
```

```
    },
    'source_4_inquisicion': {
        'pdf': SCANS_DIR / 'ES.28079.AHN_INQUISICIO_N_1667_Exp.12 _ 1640.pdf',
        'gt_transcription': TRANS_DIR / 'ES.28079.AHN.INQUISICIO_N_1667_Exp.12 _ 1640_transcription.docx',
        'language': 'Spanish',
        'century': '17th'
    },
    'source_5_1606': {
        'pdf': SCANS_DIR / 'AHPG-GPAH AU61_2 _ 1606.pdf',
        'gt_transcription': TRANS_DIR / 'AHPG-GPAH AU61.2 _ 1606_transcription.docx',
        'language': 'Spanish',
        'century': '17th'
    }
}

# Verify
print('Verifying file paths...')
all_ok = True
for source_id, config in SOURCES.items():
    pdf_ok   = config['pdf'].exists()
    trans_ok = config['gt_transcription'].exists()
    if not (pdf_ok and trans_ok):
        all_ok = False
    print(f'  {source_id}: PDF={pdf_ok} | Transcription={trans_ok}')

print('\nAll files found.' if all_ok else '\nWARNING: Some files missing.')
```

```
Verifying file paths...
  source_1_1857: PDF=True | Transcription=True
  source_2_1744: PDF=True | Transcription=True
  source_3_pleito: PDF=True | Transcription=True
  source_4_inquisicion: PDF=True | Transcription=True
  source_5_1606: PDF=True | Transcription=True

All files found.
```

In [5]:

```
# ============================================================
# UTILITY FUNCTIONS
# ============================================================

def pdf_to_images(pdf_path: str, dpi: int = 150) -> List[np.ndarray]:
```

```python
    """Convert PDF pages to numpy arrays."""
    doc = fitz.open(str(pdf_path))
    images = []
    for page in doc:
        mat = fitz.Matrix(dpi/72, dpi/72)
        pix = page.get_pixmap(matrix=mat)
        img = np.frombuffer(pix.samples, dtype=np.uint8)
        img = img.reshape(pix.height, pix.width, pix.n)
        if pix.n == 4:
            img = cv2.cvtColor(img, cv2.COLOR_RGBA2RGB)
        images.append(img)
    doc.close()
    return images


def numpy_to_pil(img: np.ndarray) -> Image.Image:
    """Convert numpy array to PIL Image."""
    if len(img.shape) == 2:
        return Image.fromarray(img, mode='L')
    return Image.fromarray(img)


def resize_for_vlm(pil_img: Image.Image,
                   max_side: int = 1280) -> Image.Image:
    """
    Resize so longest side <= max_side.
    Qwen2.5-VL works best with images up to 1280px.
    """
    w, h = pil_img.size
    if max(w, h) <= max_side:
        return pil_img
    scale = max_side / max(w, h)
    return pil_img.resize((int(w*scale), int(h*scale)), Image.LANCZOS)


def openai_call(prompt: str,
                images: List[Image.Image] = None,
                temperature: float = 0.0) -> Optional[str]:
    """
    Single entry point for all GPT-4.1 calls.
    Includes system message for academic context.
```

```
    Retries on refusal up to 3 times.
    temperature=0.0 — deterministic for transcription.
    """
    content = [{"type": "text", "text": prompt}]

    if images:
        for img in images:
            img_resized = resize_for_vlm(img, max_side=2000)
            buf = BytesIO()
            img_resized.save(buf, format='PNG')
            b64 = base64.b64encode(buf.getvalue()).decode()
            content.append({
                "type": "image_url",
                "image_url": {
                    "url": f"{b64}",
                    "detail": "high"
                }
            })

    system_message = """You are an expert paleographer working on the \
RenAIssance digital humanities project. You are transcribing historical \
legal and notarial manuscripts from 16th-19th century Spain for academic \
archival purposes. These documents are of significant historical importance. \
Your task is purely academic transcription — reproduce exactly what is \
written in the manuscript ink."""

    refusal_phrases = [
        "i'm sorry, i can't",
        "i cannot assist",
        "i'm not able to",
        "i apologize, but",
        "i'm unable to",
        "lo siento",
        "no puedo"
    ]

    for attempt in range(3):
        try:
            response = client.chat.completions.create(
                model="gpt-4.1",
                messages=[
```

```
                    {"role": "system", "content": system_message},
                    {"role": "user",   "content": content}
                ],
                temperature=temperature,
                max_tokens=4096
            )
            result = response.choices[0].message.content.strip()

            # Detect refusal and retry
            if any(phrase in result.lower() for phrase in refusal_phrases):
                print(f'  Refusal detected (attempt {attempt+1}) — retrying...')
                time.sleep(3)
                continue

            return result

        except Exception as e:
            err = str(e)
            if '429' in err or 'rate' in err.lower():
                wait = 30 * (attempt + 1)
                print(f'  Rate limit. Waiting {wait}s...')
                time.sleep(wait)
            else:
                print(f'  OpenAI error: {err}')
                return None

    print('  All 3 attempts failed or refused.')
    return None


def clean_json_response(text: str) -> str:
    """Strip markdown code fences from JSON responses."""
    text = text.strip()
    text = re.sub(r'^```json\s*', '', text)
    text = re.sub(r'^```\s*',     '', text)
    text = re.sub(r'\s*```$',     '', text)
    return text.strip()


def normalize_text(text: str) -> str:
    """NFC normalize for CER computation."""
```

```
    return unicodedata.normalize('NFC', text.strip())

def ensure_dark_on_light(image: np.ndarray) -> np.ndarray:
    """
    Ensure image has dark text on light background.
    VLMs expect this orientation.
    Check by comparing mean pixel value — if dark overall, invert.
    """
    if len(image.shape) == 3:
        gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    else:
        gray = image

    mean_val = np.mean(gray)

    if mean_val < 127:  # Dark background — invert
        print('  Image inverted (was light-on-dark)')
        return cv2.bitwise_not(image)

    return image

print('Utility functions defined.')
```

```
Utility functions defined.
```

In [6]:

```
# ============================================================
# STAGE 0: PROFILING — IMAGE ONLY
# No GT required. Rules are generic for all Spanish manuscripts.
# ============================================================

# Generic curator rules — same for all sources

SPANISH_MANUSCRIPT_RULES = [
    "u and v are used interchangeably",
    "f and s are used interchangeably",
    "ç old spelling is always modern z",
    "accents are inconsistent — transcribe exactly as written",
    "horizontal cap over letter means n follows",
    "capped q means ue follows",
    "long-s and round-s both present — transcribe as written",
    "abbreviation marks: transcribe the expansion not the symbol"
```

```
]


def build_manuscript_profile(source_id: str,
                             page_image: np.ndarray,
                             source_config: dict) -> dict:
    """
    Build scribal profile from image alone.
    No GT required.
    Generic curator rules applied to all sources.
    """


    print(f'  Building profile for {source_id}...')
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    print(f'Output directory: {OUTPUT_DIR}')
    pil_img = resize_for_vlm(numpy_to_pil(page_image), max_side=1280)

    prompt = f"""You are an expert paleographer examining a \
{source_config['century']} century {source_config['language']} \
handwritten manuscript page.

Analyze the visual characteristics of this handwriting carefully.

Return ONLY valid JSON:
{{
    "script_style": "precise description of the hand type",
    "letter_formations": {{
        "u_v": "can you visually distinguish u and v?",
        "f_s": "can you visually distinguish f and s?",
        "s":   "long-s or round-s or both?",
        "r":   "rotunda or straight?",
        "a":   "how is a formed?",
        "e":   "how is e formed?"
    }},
    "abbreviation_inventory": [
        {{
            "symbol": "the abbreviated form seen in the ink",
            "expansion": "the likely full word",
            "example": "example from the page if visible"
        }}
```

```
    ],
    "ink_quality": "consistent/variable/faded",
    "show_through_severity": "none/mild/severe",
    "marginalia_position": "describe position of any marginal notes",
    "marginalia_style": "same hand or different hand",
    "has_dropcap": true or false,
    "dropcap_description": "describe if present or null",
    "line_density": "estimated number of lines on this page as integer",
    "word_spacing": "tight/normal/loose",
    "special_challenges": ["list specific difficulties in this hand"]
}}"""

    raw          = openai_call(prompt, images=[pil_img], temperature=0.2)
    hand_profile = None

    if raw:
        try:
            hand_profile = json.loads(clean_json_response(raw))
            print(f'  Hand profile built successfully')
        except Exception as e:
            print(f'  Profile parse failed: {e} — using fallback')

    if hand_profile is None:
        hand_profile = {
            'script_style':        f'{source_config["century"]} century '
                                   f'{source_config["language"]} cursive',
            'letter_formations':   {
                'u_v': 'interchangeable',
                'f_s': 'interchangeable',
                's':   'long-s and round-s both present',
                'r':   'standard cursive',
                'a':   'standard cursive',
                'e':   'standard cursive'
            },
            'abbreviation_inventory': [],
            'ink_quality':          'variable',
            'show_through_severity':'mild',
            'marginalia_position':  'left and right margins',
            'marginalia_style':     'same hand',
            'has_dropcap':          False,
            'dropcap_description':  None,
```

```
            'line_density':         25,
            'word_spacing':         'normal',
            'special_challenges':   ['historical cursive script']
        }

    # Inject generic rules — same for all sources
    hand_profile['curator_rules'] = SPANISH_MANUSCRIPT_RULES

    profile = {
        'source_id':   source_id,
        'language':    source_config['language'],
        'century':     source_config['century'],
        'hand_profile': hand_profile
    }

    # Save to disk
    out_path = OUTPUT_DIR / f'{source_id}_profile.json'
    with open(out_path, 'w', encoding='utf-8') as f:
        json.dump(profile, f, indent=2, ensure_ascii=False)
    print(f'  Saved → {out_path.name}')

    return profile
```

In [7]:

```
# ============================================================
# STAGE 1: IMAGE PREPROCESSING
# Per page. Parameters driven by source profile.
# LLM validates output before pipeline continues.
# ============================================================

def get_preprocessing_params(profile: dict) -> dict:
    """
    Read profile to set preprocessing parameters.
    Not hardcoded — each source gets different settings.
    """
    hand         = profile['hand_profile']
    show_through = hand.get('show_through_severity', 'mild')
    ink          = hand.get('ink_quality', 'variable')

    params = {
        'sauvola_window':      25,
```

```
        'sauvola_k':           0.2,
        'clahe_clip':          2.0,
        'denoise_h':           10,
        'remove_show_through': False
    }

    if show_through == 'severe':
        params['remove_show_through'] = True
        params['sauvola_window']      = 35
        params['sauvola_k']           = 0.15

    if show_through == 'mild':
        params['sauvola_window'] = 29

    if 'fad' in ink.lower():
        params['clahe_clip'] = 4.0

    return params

def remove_show_through(image: np.ndarray) -> np.ndarray:
    """Suppress ink bleed-through via morphological background estimation."""
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY) \
        if len(image.shape) == 3 else image
    kernel     = cv2.getStructuringElement(cv2.MORPH_RECT, (50, 50))
    background = cv2.morphologyEx(gray, cv2.MORPH_CLOSE, kernel)
    corrected  = cv2.subtract(background, gray)
    corrected  = cv2.normalize(corrected, None, 0, 255, cv2.NORM_MINMAX)
    return corrected


def preprocess_page(image: np.ndarray,
                    params: dict) -> Tuple[np.ndarray, np.ndarray]:
    """
    Full preprocessing pipeline — NO deskew.
    Historical manuscripts have enough natural variation that
    deskew causes more harm than good.
    """
    # Grayscale directly — no rotation
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY) \
        if len(image.shape) == 3 else image.copy()
```

```
    # Show-through removal if needed
    if params['remove_show_through']:
        gray = remove_show_through(image)

    # CLAHE contrast enhancement
    clahe    = cv2.createCLAHE(
        clipLimit=params['clahe_clip'],
        tileGridSize=(8, 8)
    )
    enhanced = clahe.apply(gray)

    # Sauvola binarization
    thresh  = threshold_sauvola(
        enhanced,
        window_size=params['sauvola_window'],
        k=params['sauvola_k']
    )
    binary  = (enhanced > thresh).astype(np.uint8) * 255

    # Denoise
    denoised = cv2.fastNlMeansDenoising(
        binary, h=params['denoise_h']
    )

    return denoised, enhanced


def llm_validate_preprocessing(original: np.ndarray,
                               preprocessed: np.ndarray,
                               profile: dict) -> dict:
    """
    Preprocessing validation — simplified.
    We no longer use preprocessed image for transcription.
    Preprocessing only used for layout bbox detection.
    Always proceed.
    """
    return {
        'preprocessing_adequate': True,
        'issues': [],
        'recommendation': 'proceed'
    }
```

```python
def preprocess_with_validation(page_image: np.ndarray,
                               profile: dict) -> Tuple[np.ndarray, np.ndarray]:
    """
    Preprocessing for layout masking and dropcap detection only.
    NOT used for VLM transcription — original scan is passed directly.
    """
    params = get_preprocessing_params(profile)

    binary, enhanced = preprocess_page(page_image, params)

    # LLM validates
    validation = llm_validate_preprocessing(page_image, binary, profile)

    if not validation.get('preprocessing_adequate', True):
        print(f'  Preprocessing inadequate: {validation.get("issues", [])}')
        print(f'  Retrying with relaxed parameters...')

        # Relax parameters
        params['sauvola_k']           = 0.1
        params['clahe_clip']          = 1.5
        params['remove_show_through'] = False

        binary, enhanced = preprocess_page(page_image, params)
        print(f'  Retry complete.')
    else:
        issues = validation.get('issues', [])
        if issues:
            print(f'  Preprocessing adequate. Minor issues: {issues}')
        else:
            print(f'  Preprocessing validated.')

    return binary, enhanced
```

In [8]:

```python
# ============================================================
# STAGE 2: SEMANTIC LAYOUT MASKING
# VLM identifies main text bounding box.
# Everything outside is whited out before transcription.
# Kraken is NOT used — VLM defines the clean zone directly.
```

```python
# ============================================================

def vlm_get_main_text_bbox(page_image: np.ndarray,
                           profile: dict) -> Optional[dict]:
    """
    Ask VLM to identify the main text body bounding box.
    Returns coordinates as fractions of image dimensions (0.0 to 1.0).
    """
    hand         = profile['hand_profile']
    margin_pos   = hand.get('marginalia_position', 'left and right margins')
    margin_style = hand.get('marginalia_style', 'same hand')
    line_density = hand.get('line_density', '25')

    pil_img = resize_for_vlm(numpy_to_pil(page_image), max_side=768)

    prompt = f"""You are an expert paleographer and digital archivist specializing in {profile['century']} century

TASK:
Identify the bounding box for the 'Primary Record Block'. This includes all text and formal marks written as part o

DEFINITION OF MAIN TEXT (INCLUDE):
1. ANCHORING ELEMENTS: Any text at the top that identifies the record (e.g., reference numbers, dates, locations, t
2. FORMULAIC INVOCATIONS: Formal openings (religious or legal) that often appear above the main body.
3. DECORATIVE INITIALS (DROPCAPS): Large, ornate, or oversized starting letters. These are part of the first word a
4. INTEGRAL ASYMMETRY: If a header or 'Item' marker is positioned further left than the main text block, the bounda
5. NOTARIAL MARKS: Complex signatures, rubrics, or symbols (often at the bottom) that certify the document.
6. INTERLINEAR NOTATIONS: Small words or caret-insertions squeezed between lines.

DEFINITION OF MARGINALIA (EXCLUDE):
1. EXTERNAL SUMMARIES: Short notes in the far outer margins used as a 'table of contents' or index for the page.
2. LATER ANNOTATIONS: Metadata added by modern archivists (pencil marks, library stamps).
3. PAGE/FOLIO NUMBERS: Digits at the extreme corners.
4. BINDING ARTIFACTS: Shadows or text 'bleeding' from the reverse side.

COORDINATE GUIDANCE:
- Do NOT crop based on paragraph justification.
- Use the left-most character (header/ID) and the right-most character to define width.
- Use the top-most character (Invocation/Date) and the bottom-most mark (Signature/Rubric) to define height.

Return ONLY valid JSON:
{{
```

```
    "x_min": 0.0,
    "y_min": 0.0,
    "x_max": 1.0,
    "y_max": 1.0,
    "confidence": "high/medium/low",
    "notes": "Specifically mentions if dropcaps or headers were found and preserved."
}}"""

    raw = openai_call(prompt, images=[pil_img], temperature=0.0)

    if not raw:
        print('  VLM bbox failed — using full page')
        return None

    try:
        bbox = json.loads(clean_json_response(raw))

        # Validate bbox values are sensible
        x_min = float(bbox.get('x_min', 0.0))
        y_min = float(bbox.get('y_min', 0.0))
        x_max = float(bbox.get('x_max', 1.0))
        y_max = float(bbox.get('y_max', 1.0))

        # Sanity checks
        if x_min >= x_max or y_min >= y_max:
            print('  Invalid bbox (min >= max) — using full page')
            return None

        if (x_max - x_min) < 0.3 or (y_max - y_min) < 0.3:
            print('  Bbox too small (<30% of page) — using full page')
            return None

        bbox['x_min'] = x_min
        bbox['y_min'] = y_min
        bbox['x_max'] = x_max
        bbox['y_max'] = y_max

        print(f'  Main text bbox: '
              f'x=[{x_min:.2f}, {x_max:.2f}] '
              f'y=[{y_min:.2f}, {y_max:.2f}] '
              f'conf={bbox.get("confidence", "?")}')
```

```
        if bbox.get('notes'):
            print(f'  Excluded: {bbox["notes"]}')

        return bbox

    except (json.JSONDecodeError, KeyError, ValueError) as e:
        print(f'  Bbox parse failed: {e}')
        print(f'  Raw preview: {raw[:200]}')
        return None


def apply_layout_mask(page_image: np.ndarray,
                      bbox: Optional[dict]) -> np.ndarray:
    """
    White out everything outside the main text bbox.
    Add padding to avoid cutting edge characters.
    """
    if bbox is None:
        return page_image

    h, w   = page_image.shape[:2]
    masked = np.ones_like(page_image) * 255

    # Add 2% padding on all sides to avoid cutting edge text
    x1 = max(0, int((bbox['x_min'] - 0.02) * w))
    y1 = max(0, int((bbox['y_min'] - 0.02) * h))
    x2 = min(w, int((bbox['x_max'] + 0.02) * w))
    y2 = min(h, int((bbox['y_max'] + 0.02) * h))

    masked[y1:y2, x1:x2] = page_image[y1:y2, x1:x2]
    return masked


def get_masked_page(page_image: np.ndarray,
                    profile: dict) -> Tuple[np.ndarray, Optional[dict]]:
    """
    Full layout masking pipeline for one page.
    Returns: (masked image, bbox dict)
    """
    bbox   = vlm_get_main_text_bbox(page_image, profile)
```

```
    masked = apply_layout_mask(page_image, bbox)
    return masked, bbox
```

In [9]:

```
# ============================================================
# STAGE 3: DROPCAP ISOLATION
# Detect large initial letter, identify it with VLM,
# mask it out before transcription, reintegrate after.
# Only runs if profile says has_dropcap = True.
# ============================================================
def process_dropcap(page_image: np.ndarray,
                    profile: dict) -> Tuple[np.ndarray, Optional[str]]:
    """
    Simplified dropcap handling.
    Do NOT use connected components — too unreliable on manuscripts.
    Instead ask VLM directly what the first letter is.
    The full page transcription (Stage 4) will handle it in context.
    We just record the letter for reintegration if needed.
    """
    hand_profile = profile['hand_profile']

    if not hand_profile.get('has_dropcap', False):
        print('  No dropcap in this source — skipping.')
        return page_image, None

    # Ask VLM what the dropcap letter is — no masking, no crop
    # Just show the full page and ask
    pil_img = resize_for_vlm(numpy_to_pil(page_image), max_side=768)
    desc    = hand_profile.get('dropcap_description', 'large initial letter')

    prompt = f"""You are a paleographer analyzing a {profile['century']} century \
{profile['language']} manuscript page.

TASK:
Identify if the document begins with a 'Dropcap' (an oversized, ornate initial letter).

GUIDELINES:
1. OVERSIZED INITIAL: Look at the very first letter of the main text body (after the headers).
2. IS IT A DROPCAP?: If the first letter is significantly larger than the surrounding text \
(spanning 2 or more lines), identify it.
3. NO DROPCAP: If the first letter is a standard-sized capital or only slightly larger, \
```

```
return has_dropcap as false and letter as "none".

Return ONLY valid JSON:
{{
    "has_dropcap": true,
    "letter": "the single capital letter or none",
    "first_word": "the complete word starting with that letter",
    "confidence": "high/medium/low",
    "reasoning": "e.g. Large ornate E spanning 3 lines or Standard capital E"
}}"""

    raw = openai_call(prompt, images=[pil_img], temperature=0.0)

    dropcap_char = None
    if raw:
        try:
            result       = json.loads(clean_json_response(raw))

            # Check if VLM agrees there is a dropcap
            vlm_has_dropcap = result.get('has_dropcap', False)
            if not vlm_has_dropcap:
                print('  VLM confirms no dropcap on this page.')
                return page_image, None

            dropcap_char = result.get('letter', '?').strip().upper()
            if dropcap_char == 'NONE' or not dropcap_char:
                return page_image, None
            if len(dropcap_char) > 1:
                dropcap_char = dropcap_char[0]

            confidence = result.get('confidence', 'low')
            first_word = result.get('first_word', '')
            reasoning  = result.get('reasoning', '')

            print(f'  Dropcap: "{dropcap_char}" '
                  f'(conf={confidence})')
            print(f'  First word: "{first_word}"')
            print(f'  Reasoning: "{reasoning[:80]}"')

        except Exception as e:
            print(f'  Dropcap parse failed: {e}')
```

```
            dropcap_char = None

    return page_image, dropcap_char
```

In [10]:

```
# ============================================================
# STAGE 4: FULL PAGE VLM TRANSCRIPTION
# Uses enhanced grayscale (not binary) for better VLM reading.
# Splits page into top/bottom halves to prevent hallucination.
# ============================================================

def split_page_halves(image: np.ndarray,
                      overlap: float = 0.05) -> Tuple[np.ndarray, np.ndarray]:
    """
    Split page into top and bottom halves with small overlap.
    Overlap prevents losing text at the split boundary.
    """
    h = image.shape[0]
    mid        = h // 2
    overlap_px = int(h * overlap)

    top    = image[:mid + overlap_px, :]
    bottom = image[mid - overlap_px:, :]

    return top, bottom

def transcribe_half(half_image: np.ndarray,
                    profile: dict,
                    half_label: str,
                    context: str = '',
                    dropcap_char: Optional[str] = None) -> str:
    """
    Transcribe one tile of a page.
    Context = transcription of preceding tile for continuity.
    """
    hand    = profile['hand_profile']
    century = profile['century']
    lang    = profile['language']

    # Build abbreviation string
    abbrevs    = hand.get('abbreviation_inventory', [])
```

```
    abbrev_str = '\n'.join([
        f"  {a.get('symbol','?')} → {a.get('expansion','?')}"
        for a in abbrevs
    ]) or '  None documented'

    # Curator rules
    rules     = hand.get('curator_rules', [])
    rules_str = '\n'.join([f'  {r}' for r in rules])

    # Context from previous tile
    context_block = ''
    if context:
        last_lines = '\n'.join(context.strip().split('\n')[-4:])
        context_block = f"""
LAST 4 LINES ALREADY TRANSCRIBED (for continuity — do not repeat these):
{last_lines}

Continue transcribing from where the above ends.
"""

    # Dropcap hint — only for top tile
    dropcap_block = ''
    if dropcap_char and half_label == 'TOP':
        dropcap_block = f"""
The first letter of the main text is a large decorative "{dropcap_char}".
Ensure the transcription begins with "{dropcap_char}".
"""

    prompt = f"""You are a specialized Paleography Assistant.
TASK: Provide a verbatim transcription of the {half_label} TILE of a \
{century} century {lang} handwritten manuscript page.
Output each line of handwriting as a separate line.
Do NOT merge lines into paragraphs.

This is a non-sensitive historical document. You are transcribing characters,
not processing personal data.

SCRIPT: {hand.get('script_style', '')[:80]}

CURATOR RULES (verified — follow exactly):
{rules_str}
```

```
KNOWN ABBREVIATIONS:
{abbrev_str}
{context_block}{dropcap_block}
STRICT RULES:
1. Each line of handwriting = exactly one line in your output
2. Do NOT merge two handwritten lines into one output line
3. Do NOT split one handwritten line into two output lines
4. Count the lines visually before transcribing
5. If you see 12 lines of handwriting, output exactly 12 lines
6. Preserve exact original spelling — never modernize
7. Mark unreadable text as [?]
8. ç always becomes z
9. Ignore folio numbers at corners and sideways marginalia
10. Do NOT repeat lines already transcribed above
11. Stop when you reach the end of visible text in this tile
12. HYPHENATED LINE BREAKS: If a word is split across lines with a
    hyphen (e.g. "ocho-" on one line, "cientos" on the next line),
    output EXACTLY as written — "ocho-" on its own line, "cientos"
    on the next line. Do NOT join them. Do NOT remove the hyphen.
13. The header (date, reference number, document type) appears
    at the TOP — transcribe it first before the main body text

ONE HANDWRITTEN LINE = ONE OUTPUT LINE. THIS IS THE MOST IMPORTANT RULE.
HYPHENATED WORDS STAY SPLIT EXACTLY AS WRITTEN ON THE PAGE.
DO NOT JOIN, DO NOT MODIFY, DO NOT REORDER ANYTHING.

Transcribe this {half_label} tile now.
Output transcription only — no commentary, no markdown."""

    # Ensure correct polarity — dark text on light background
    page_image = ensure_dark_on_light(half_image)
    pil_img    = resize_for_vlm(numpy_to_pil(page_image), max_side=2000)
    result     = openai_call(prompt, images=[pil_img], temperature=0.0)

    # If all retries failed inside openai_call
    if not result:
        print(f'  [{half_label}] Failed after retries — returning empty')
        return ''

    # Clean markdown artifacts
```

```python
    result = re.sub(r'^```[a-zA-Z]*\n?', '', result)
    result = re.sub(r'\n?```$', '', result)
    result = result.strip()

    return result


def check_hallucination(text: str, threshold: int = 3) -> bool:
    """
    Detect repetition hallucination.
    Returns True if hallucination detected.
    """
    if not text:
        return False
    lines  = [l.strip() for l in text.split('\n') if l.strip()]
    if len(lines) < 5:
        return False
    counts = Counter(lines)
    _, max_count = counts.most_common(1)[0]
    if max_count >= threshold:
        print(f'  HALLUCINATION DETECTED: line repeated {max_count}x')
        return True
    return False


def merge_halves(texts: List[str]) -> str:
    """
    Fuzzy-matches overlapping lines to stitch multiple tiles.
    Replaces the old 2-half merge to prevent data loss.
    """
    if not texts: return ""

    # Start with the lines from the first (TOP) tile
    final_lines = [l.strip() for l in texts[0].split('\n') if l.strip()]

    for i in range(1, len(texts)):
        current_lines = [l.strip() for l in texts[i].split('\n') if l.strip()]
        match_idx = -1

        # Look for the last 8 lines of our progress in the new tile
        for j in range(len(current_lines)):
```

```
            line = current_lines[j].lower()
            if len(line) < 10: continue # Skip short fragments for matching

            # Check against the tail of our current document
            for k in range(max(0, len(final_lines)-10), len(final_lines)):
                if cer(final_lines[k].lower(), line) < 0.30: # 70% similarity match
                    match_idx = j
                    break
            if match_idx != -1: break

        # Append only the new lines starting after the match point
        final_lines.extend(current_lines[match_idx+1:])

    return '\n'.join(final_lines)

def transcribe_page(page_image: np.ndarray,
                    profile: dict,
                    dropcap_char: Optional[str] = None,
                    preceding_text: str = '') -> str:
    """
    Full page transcription using a 3-tile sliding window.
    Fixes the 'missing half' issue by ensuring overlapping coverage.
    """
    h, w = page_image.shape[:2]
    print('  Slicing page into 3 overlapping tiles...')

    # Define 3 zones: Top 45%, Middle 45%, Bottom 45%
    tile_h = int(h * 0.45)

    tiles = [
        ('TOP', page_image[0:tile_h, :]),
        ('MID', page_image[int(h*0.28):int(h*0.73), :]),
        ('BOT', page_image[h-tile_h:h, :])
    ]

    tile_results = []
    current_context = preceding_text

    for label, tile_img in tiles:
        print(f'  Transcribing {label} tile...')
        # Reuse your existing transcribe_half function
```

```
        text = transcribe_half(
            tile_img, profile,
            half_label=label,
            context=current_context,
            dropcap_char=dropcap_char if label == 'TOP' else None
        )
        tile_results.append(text)
        current_context = text # Pass context to next tile for continuity

    # Use the new multi-tile merge logic
    merged = merge_halves(tile_results)
    print(f'  Merged transcription: {len(merged.splitlines())} lines')

    return merged
```

In [11]:

```
#Correction Function
def llm_spelling_correction(transcription: str,
                            profile: dict,
                            page_image: np.ndarray = None) -> Tuple[str, List[dict]]:
    """
    Stage 7: LLM predicts most likely correct spelling
    and recovers any missing lines by cross-referencing image.
    """
    hand    = profile['hand_profile']
    century = profile['century']
    lang    = profile['language']

    rules_str  = '\n'.join([f'  {r}' for r in hand.get('curator_rules', [])])
    abbrev_str = '\n'.join([
        f"  {a.get('symbol','?')} → {a.get('expansion','?')}"
        for a in hand.get('abbreviation_inventory', [])
    ]) or '  None documented'

    prompt = f"""You are an expert paleographer correcting a draft \
transcription of a {century} century {lang} manuscript.

CURATOR RULES:
{rules_str}

KNOWN ABBREVIATIONS:
```

```
{abbrev_str}

DRAFT TRANSCRIPTION:
{transcription}

YOUR TASKS:
1. MISSING LINES: Compare the draft against the image carefully.
   If any handwritten lines are present in the image but missing
   from the draft — add them in the correct position.
   This is the most important task.

2. SPELLING CORRECTION: For each [?] marker and each clearly
   garbled word, predict the most likely correct {lang} spelling
   based on surrounding context and {century} century legal vocabulary.

3. LETTER CONFUSIONS: Fix obvious confusion patterns:
   - u/v interchangeable
   - f/s interchangeable
   - m/n/ni confusion
   - rr/r confusion at line start
   - El/M confusion at line start (capital letters)

HARD CONSTRAINTS:
- Do NOT join hyphenated words across lines
- "ocho-" on one line and "cientos" on next line MUST stay as two lines
- NEVER merge two lines into one
- NEVER reduce the line count
- Line count in = line count out
- Do NOT modernize any spelling
- Do NOT add lines that are not visible in the image
- Do NOT remove any lines
- Do NOT reorder any lines
- Preserve all line breaks exactly
- One handwritten line = one output line
- If you cannot confidently read a word — leave as [?]
- Do NOT change proper names unless they contain [?]
- Do NOT standardize spelling of surnames across lines


Return ONLY valid JSON:
{{
```

```
    "corrected_transcription": "full corrected text preserving all line breaks",
    "corrections": [
        {{
            "type": "missing_line/spelling/garbled",
            "original": "what it was or empty if missing",
            "corrected": "what it became",
            "confidence": "high/medium/low",
            "reasoning": "brief reason"
        }}
    ]
}}"""

    # Pass image if available — enables missing line detection
    images = None
    if page_image is not None:
        page_clean = ensure_dark_on_light(page_image)
        pil_img    = resize_for_vlm(numpy_to_pil(page_clean), max_side=2000)
        images     = [pil_img]
        print('  Correction using image + text...')
    else:
        print('  Correction using text only...')

    raw = openai_call(prompt, images=images, temperature=0.0)

    if not raw:
        print('  Spelling correction failed — returning original')
        return transcription, []

    try:
        result      = json.loads(clean_json_response(raw))
        corrected   = result.get('corrected_transcription', transcription)
        corrections = result.get('corrections', [])

        print(f'  Corrections made: {len(corrections)}')
        if corrections:
            print('  Sample corrections:')
            for c in corrections[:5]:
                print(f'    [{c.get("type","?")}|{c.get("confidence","?")}] '
                      f'"{c.get("original","")}" → '
                      f'"{c.get("corrected","")}" '
                      f'({c.get("reasoning","")[:50]})')
```

```
        return corrected, corrections

    except json.JSONDecodeError as e:
        print(f'  JSON parse failed: {e}')
        # Try regex recovery
        match = re.search(
            r'"corrected_transcription"\s*:\s*"(.*?)"(?=\s*,\s*"corrections")',
            raw, re.DOTALL
        )
        if match:
            corrected = match.group(1).replace('\\n', '\n')
            print('  Recovered from partial JSON')
            return corrected, []
        return transcription, []
```

In [12]:

```
# ============================================================
# STAGE 6: DETERMINISTIC POST-PROCESSING RULES
# Runs AFTER coherence check.
# Rules have final word — cannot be overridden by LLM.
# ============================================================

def apply_deterministic_rules(text: str, profile: dict) -> str:
    """
    Hard rule-based corrections.
    Final stage — nothing can override these.
    """
    hand_profile = profile.get('hand_profile', {})
    changes      = []

    # ------------------------------------------------------
    # Rule 1: ç → z (curator rule — always)
    # ------------------------------------------------------
    if 'ç' in text or 'Ç' in text:
        text = text.replace('ç', 'z').replace('Ç', 'Z')
        changes.append('ç → z')

    # ------------------------------------------------------
    # Rule 2: Tilde expansions
    # ------------------------------------------------------
```

```
    if re.search(r'q[~]', text):
        text = re.sub(r'q[~]', 'que', text)
        changes.append('q~ → que')

    if re.search(r'n[~]', text):
        text = re.sub(r'n[~]', 'ñ', text)
        changes.append('n~ → ñ')

    if re.search(r'([aeiouáéíóúAEIOU])[~]', text):
        text = re.sub(r'([aeiouáéíóúAEIOU])[~]', r'\1n', text)
        changes.append('vowel~ → vowel+n')

    # ---------------------------------------------------------
    # Rule 3: Macron expansions
    # ---------------------------------------------------------
    macron_map = {
        'ā': 'an', 'ē': 'en', 'ī': 'in',
        'ō': 'on', 'ū': 'un',
        'Ā': 'An', 'Ē': 'En', 'Ō': 'On'
    }
    for char, expansion in macron_map.items():
        if char in text:
            text = text.replace(char, expansion)
            changes.append(f'{char} → {expansion}')

    # ---------------------------------------------------------
    # Rule 5: Clean up blank lines
    # ---------------------------------------------------------
    text = re.sub(r'\n{3,}', '\n\n', text)
    text = text.strip()

    if changes:
        print(f'  Rules applied: {", ".join(changes)}')
    else:
        print('  No rule changes needed')

    return text
def detect_columns(page_image: np.ndarray) -> int:
    """
    Ask GPT-4.1 how many columns this page has.
    Returns 1 or 2.
```

```
    """
    pil_img = resize_for_vlm(numpy_to_pil(page_image), max_side=1280)
    prompt  = """Analyze this manuscript page layout.
How many columns of text are present?
Return ONLY valid JSON: {"columns": 1}"""

    raw = openai_call(prompt, images=[pil_img], temperature=0.0)
    try:
        result = json.loads(clean_json_response(raw))
        return result.get('columns', 1)
    except Exception:
        return 1


def split_columns(page_image: np.ndarray) -> List[np.ndarray]:
    """
    Split page into left and right column images.
    Simple vertical split at midpoint.
    """
    w = page_image.shape[1]
    left  = page_image[:, :w//2]
    right = page_image[:, w//2:]
    return [left, right]
```

In [13]:

```
# ============================================================
# STAGE 7: REFINED EVALUATION
# ============================================================

def load_gt_for_evaluation(gt_path: str) -> dict:
    path = Path(gt_path)

    if path.suffix == '.docx':
        from docx import Document
        doc     = Document(str(path))
        content = '\n'.join([p.text for p in doc.paragraphs])
    else:
        with open(path, 'r', encoding='utf-8') as f:
            content = f.read()

    notes       = ''
```

```
    notes_match = re.search(
        r'NOTES?:?\s*(.*?)(?=PDF\s*p\.?\s*\d+)',
        content, re.DOTALL | re.IGNORECASE
    )
    if notes_match:
        notes = notes_match.group(1).strip()

    page_markers = list(re.finditer(
        r'PDF\s*p\.?\s*(\d+)',
        content, re.IGNORECASE
    ))

    pages = {}
    for i, marker in enumerate(page_markers):
        page_num = int(marker.group(1))
        start    = marker.end()
        end      = page_markers[i+1].start() \
                   if i+1 < len(page_markers) else len(content)

        page_text = content[start:end].strip()

        # Remove blank lines — GT should be line-by-line with no gaps
        lines = [l.strip() for l in page_text.split('\n')
                 if l.strip()]  # <-- this removes blank lines

        pages[page_num] = {
            'text':  '\n'.join(lines),
            'lines': lines
        }

    print(f'  GT pages found : {sorted(pages.keys())}')
    for pn, pd in pages.items():
        print(f'  Page {pn}        : {len(pd["lines"])} lines')

    return {
        'notes':    notes,
        'pages':    pages,
        'gt_text':  pages.get(1, {}).get('text', ''),
        'gt_lines': pages.get(1, {}).get('lines', [])
    }
```

```python
def calculate_strict_cer(prediction: str,
                         ground_truth: str) -> float:
    def final_norm(t):
        t = normalize_text(t)
        # Remove blank lines — match GT structure
        lines = [l.strip() for l in t.split('\n') if l.strip()]
        return '\n'.join(lines)

    p = final_norm(prediction)
    g = final_norm(ground_truth)

    if len(g) < 5:
        print('  WARNING: Ground Truth appears truncated or empty.')
        return 1.0

    p_lines = [l for l in p.split('\n') if l.strip()]
    g_lines = [l for l in g.split('\n') if l.strip()]

    if len(p_lines) != len(g_lines):
        print(f'  WARNING: Line count mismatch — '
              f'GT={len(g_lines)} lines, Pred={len(p_lines)} lines')
    else:
        print(f'  Line count match: {len(p_lines)} lines ✓')

    return cer(g, p)
```

In [14]:

```python
def run_pipeline(image_path: str,
                 source_id: str = None,
                 page_num: int = 0,
                 gt_text: str = None) -> str:
    """
    Full pipeline — 3-tile GPT-4.1 transcription (no Surya).
    Stage 0 — Load or build profile
    Stage 1 — Preprocess (layout detection only)
    Stage 2 — GPT-4.1 layout masking
    Stage 3 — GPT-4.1 dropcap detection
    Stage 4 — GPT-4.1 3-tile transcription
    Stage 6 — LLM spelling correction (with image)
    Stage 7 — Deterministic rules
    """
```

```
    print(f'Running pipeline on: {image_path} (Page {page_num+1})')
    print('=' * 50)

    # Load image
    path = Path(image_path)
    if path.suffix.lower() == '.pdf':
        pages    = pdf_to_images(str(path), dpi=150)
        img_orig = pages[page_num]
    else:
        img_orig = np.array(Image.open(image_path).convert('RGB'))

    # Load or build profile
    profile_path = OUTPUT_DIR / f'{source_id}_profile.json'
    if source_id and profile_path.exists():
        with open(profile_path) as f:
            profile = json.load(f)
        print(f'Profile loaded: {source_id}')
    else:
        print('No profile found — building from image...')
        profile = build_manuscript_profile(
            source_id     = source_id or Path(image_path).stem,
            page_image    = img_orig,
            source_config = {'language': 'Spanish', 'century': '17th'}
        )

    # Stage 1 — preprocess for layout only
    binary, _ = preprocess_with_validation(img_orig, profile)

    # Stage 2 — GPT-4.1 layout masking
    print('\nStage 2: GPT-4.1 layout masking...')
    _, bbox            = get_masked_page(binary, profile)
    masked_for_reading = apply_layout_mask(img_orig, bbox)
    masked_for_reading = ensure_dark_on_light(masked_for_reading)

    # Stage 3 — GPT-4.1 dropcap detection
    print('\nStage 3: Dropcap detection...')
    _, dropcap_char = process_dropcap(masked_for_reading, profile)

    # Stage 4 — GPT-4.1 3-tile transcription
    print('\nStage 4: Transcribing...')
    # Detect columns
```

```
    n_cols = detect_columns(masked_for_reading)

    if n_cols == 2:
        print('  Two-column layout detected — splitting')
        columns      = split_columns(masked_for_reading)
        col1_text    = transcribe_page(columns[0], profile, dropcap_char=dropcap_char)
        col2_text    = transcribe_page(columns[1], profile)
        raw          = col1_text + '\n' + col2_text
    else:
        raw = transcribe_page(
            masked_for_reading, profile,
            dropcap_char=dropcap_char,
            preceding_text=''
        )

    # Stage 6 — LLM spelling correction with image
    print('\nStage 6: LLM spelling correction...')
    after_correction, _ = llm_spelling_correction(
        raw,
        profile,
        page_image=masked_for_reading
    )

    # Stage 7 — Deterministic rules (always last)
    print('\nStage 7: Applying deterministic rules...')
    final = apply_deterministic_rules(after_correction, profile)

    print('\n--- FINAL TRANSCRIPTION ---')
    print(final)

    if gt_text:
        raw_cer        = calculate_strict_cer(raw, gt_text)
        correction_cer = calculate_strict_cer(after_correction, gt_text)
        final_cer      = calculate_strict_cer(final, gt_text)
        print(f'\n--- CER SUMMARY ---')
        print(f'Stage 4 raw VLM    : {raw_cer:.3f} ({raw_cer*100:.1f}%)')
        print(f'Stage 6 LLM corr   : {correction_cer:.3f} ({correction_cer*100:.1f}%)')
        print(f'Stage 7 rules      : {final_cer:.3f} ({final_cer*100:.1f}%)')
        print(f'Total improvement  : {(raw_cer - final_cer)*100:.1f}%')
```

```
    return final
```

In [15]:
```
import json
from jiwer import wer, cer

# Storage for results
evaluation_results = []

print(f"{'Source ID':<25} | {'CER':<8} | {'WER':<8}")
print("-" * 45)

for source_id, config in SOURCES.items():
    try:
        # 1. Load Ground Truth for Evaluation
        gt_data = load_gt_for_evaluation(str(config['gt_transcription']))

        # Mapping PDF page 0 to GT Page 1
        gt_page_text = gt_data['pages'].get(1, {}).get('text', '')

        if not gt_page_text:
            print(f"  [!] Skipping {source_id}: Ground Truth for Page 1 missing.")
            continue

        # 2. Run Inference
        # The internal transcribe_half now uses the 'Public Domain' framing
        prediction = run_pipeline(
            image_path = str(config['pdf']),
            source_id  = source_id,
            page_num   = 0,
            gt_text    = gt_page_text
        )

        # 3. Normalize for Scoring
        def score_norm(t):
            t = normalize_text(t)
            t = re.sub(r'\n+', '\n', t) # Structural normalization
            return t.strip()

        pred_final = score_norm(prediction)
```

```
        gt_final   = score_norm(gt_page_text)

        # 4. Calculate Metrics
        curr_cer = cer(gt_final, pred_final)
        curr_wer = wer(gt_final, pred_final)

        # 5. Log Data
        evaluation_results.append({
            "source_id": source_id,
            "metrics": {
                "cer": round(curr_cer, 4),
                "wer": round(curr_wer, 4)
            },
            "prediction": prediction,
            "ground_truth": gt_page_text
        })

        print(f"{source_id:<25} | {curr_cer:>7.2%} | {curr_wer:>7.2%}")

    except Exception as e:
        print(f"  [X] Failed processing {source_id}: {e}")

# ============================================================
# SAVE RESULTS
# ============================================================
results_path = Path('/kaggle/working/test_result.json')
with open(results_path, 'w', encoding='utf-8') as f:
    json.dump(evaluation_results, f, indent=4, ensure_ascii=False)

# Summary Stats
if evaluation_results:
    avg_cer = sum(r['metrics']['cer'] for r in evaluation_results) / len(evaluation_results)
    print("\n" + "="*45)
    print(f"MACRO-AVERAGE CER: {avg_cer:.2%}")
    print(f"Detailed logs saved to: {results_path}")
```

```
Source ID                 | CER      | WER
-----------------------------------------------
  GT pages found : [1]
  Page 1        : 24 lines
Running pipeline on: /kaggle/input/datasets/indrasn0wal/humanai-handwriting-ocr/handwriting/text scans/PT3279_146_34
2 _ 1857.pdf (Page 1)
==================================================
No profile found — building from image...
  Building profile for source_1_1857...
Output directory: /kaggle/working/outputs
  Hand profile built successfully
  Saved → source_1_1857_profile.json
  Preprocessing validated.

Stage 2: GPT-4.1 layout masking...
  Main text bbox: x=[0.09, 0.93] y=[0.07, 0.93] conf=high
  Excluded: Bounding box includes the anchoring elements at the top left (Número, date), the decorative initial 'E'
at the start of the main text, and the notarial rubric at the bottom. The left boundary is expanded to include the l
eftmost header. No marginalia or external summaries are included. Dropcap and header are both present and preserved.

Stage 3: Dropcap detection...
  Dropcap: "E" (conf=high)
  First word: "En"
  Reasoning: "The initial 'E' at the start of the main text is significantly larger and more o"

Stage 4: Transcribing...
  Slicing page into 3 overlapping tiles...
  Transcribing TOP tile...
  Transcribing MID tile...
  Transcribing BOT tile...
  Merged transcription: 23 lines

Stage 6: LLM spelling correction...
  Correction using image + text...
  Corrections made: 4
  Sample corrections:
    [spelling|high] "cincuenta y cuatro" → "cincuenta y diez" (The manuscript clearly reads 'cincuenta y diez', n)
    [spelling|high] "Cotineguiaga" → "Cutineguiaga" (The manuscript shows 'Cutineguiaga' with a clear ')
    [spelling|high] "Fruct. Mineguiaga" → "Anto. Mineguiaga" (The manuscript abbreviates 'Anto.' for 'Antonio', )
    [spelling|high] "au bien de edad soltera" → "au bien de edad soltera" (This phrase is as written in the manuscri
pt, no co)
```

```
Stage 7: Applying deterministic rules...
  No rule changes needed

--- FINAL TRANSCRIPTION ---
Junio 9 de 1854.
D.ª copia
En esta villa de Lizarra a nueve de junio de mil ocho-
cientos cincuenta y diez, ante mi el escribano real y numeral
de ella, y Notario publico de este Obispado de Pamplona,
y competente número de testigos que al final serán nom-
brados, comparecieron personalmente de la una parte D.ª María
Marina de Aldama, viuda de D. Manuel Juan Cuti-
eguiaga, con su legitimo hijo Dn. José Anto. Mineguiaga
de estado soltero, y de la otra Dn. Juan Francisco de Min-
teguiaga viudo de Da. Juana Maria de Goyeneche
con su hija legitima Da. Manuela Josefa de Minteguiaga
au bien de edad soltera, todos ellos vecinos de esta ciudad villa
y precedidas entre los mismos las senias y licencias materna-
y paternales prevenidas por derecho de cuya peticion, con
cesion y aceptacion reciprocas hago fe yo el escribano y Notario
dijeron: Q̃ al paso que se conformaron los esposados Dn. José
Antonio y Da. Manuela Josefa Minteguiaga en contraher
en matrimonio, y obtuvieron de su Santidad la dispensacion
del parentesco del tercero con cuarto grado de consanguinidad
que media entre ellos, se dieron mútua palabra de casarse
in facie Eclesie, obligandose á que ninguno el contraeria con
otra persona sin consentimiento por escrito de el otro contrayente
  WARNING: Line count mismatch — GT=24 lines, Pred=23 lines
  WARNING: Line count mismatch — GT=24 lines, Pred=23 lines
  WARNING: Line count mismatch — GT=24 lines, Pred=23 lines

--- CER SUMMARY ---
Stage 4 raw VLM    : 0.074 (7.4%)
Stage 6 LLM corr   : 0.069 (6.9%)
Stage 7 rules      : 0.069 (6.9%)
Total improvement  : 0.5%
source_1_1857             |   6.90% |  21.35%
  GT pages found : [1]
  Page 1        : 24 lines
Running pipeline on: /kaggle/input/datasets/indrasn0wal/humanai-handwriting-ocr/handwriting/text scans/AHPG-GPAH 1_1
```

```
716_A.35 _ 1744.pdf (Page 1)
==================================================
No profile found — building from image...
  Building profile for source_2_1744...
Output directory: /kaggle/working/outputs
  Hand profile built successfully
  Saved → source_2_1744_profile.json
  Preprocessing validated.

Stage 2: GPT-4.1 layout masking...
  Main text bbox: x=[0.13, 0.97] y=[0.03, 0.97] conf=high
  Excluded: Bounding box includes the anchoring elements at the top (date, title, and reference), the main body of t
ext, and the leftmost header which is indented further left than the main block. No large decorative dropcaps are pr
esent, but the initial lines are preserved. Notarial marks or signatures at the bottom are included. Marginalia on t
he far left and external summaries are excluded.

Stage 3: Dropcap detection...
  No dropcap in this source — skipping.

Stage 4: Transcribing...
  Slicing page into 3 overlapping tiles...
  Transcribing TOP tile...
  Transcribing MID tile...
  Transcribing BOT tile...
  Merged transcription: 23 lines

Stage 6: LLM spelling correction...
  Correction using image + text...
  Corrections made: 20
  Sample corrections:
    [missing_line|high] "" → "Andres de Muguruza natal y vezino de esta Villa." (This line is present in the manuscr
ipt but missing)
    [missing_line|high] "" → "A mi con parecer Como mas aya lugar. Hazido que" (This line is present in the manuscri
pt but missing)
    [spelling|high] "de Sagaztequiçieta ya Difuntos Hazido y procreado" → "de Sagaztequiquieta ya Difuntos Hazido y
procreado" (The manuscript clearly reads 'Sagaztequiquieta', n)
    [spelling|high] "Cataliña" → "Cataliña" (Confirmed as correct from the manuscript.)
    [spelling|high] "laztequi" → "laztequi" (Confirmed as correct from the manuscript.)

Stage 7: Applying deterministic rules...
  Rules applied: ç → z
```

```
--- FINAL TRANSCRIPTION ---
año de 1744=
Informacion de filacion hidalguia y limpieza de Sagrero
curiada ante el Señor Alcalde de esta V.a del Goibar apedim.
a Andres de Muguruza y Sara depatadas a Banpo casa.
dos de Confirmacion=
Andres de Muguruza natal y vezino de esta Villa.
A mi con parecer Como mas aya lugar. Hazido que
Soy hijo Lezmo de Domingo de Muguruza y d̃ Antonia
de Sagaztequiquieta ya Difuntos Hazido y procreado
en el Matrimonio Lezmo de ellos, Nieto por parte
Paterna de Juan de Muguruza y d̃ Cataliña de Sa-
laztequi su Lezma muger y por la Materna de
Bartt. de Sagaztequiquieta y Josepha de Lanaztequi
marido y muger Lezmos todos Vecinos de esta dha
Villa quienes y Zo. somos Nobles hijos dalgo de san-
gue Christiana, Viejos sin Vara alguna de Moros,
Judios, Hereges, Penitenziados por la Santa In-
quizizion ni de otra Secta Reprobada, antes bien
dezendientes Lezitimos de los Primeros Poblado
ra desta M. N. y M. L. Provincia de Guipuz-
coa y originarios y dependientes de las Casas Sola-
res Infanzonas de Mozquuruzza Reyna, sita y nota-
ria en la Villa de la de Sagaztequiquita sita en
la Villa de Acybar, de la de Gallazcuequi, sita
en la Villa de Bezoara y de la de Lannizcuequi sita
  WARNING: Line count mismatch — GT=24 lines, Pred=23 lines
  WARNING: Line count mismatch — GT=24 lines, Pred=25 lines
  WARNING: Line count mismatch — GT=24 lines, Pred=25 lines

--- CER SUMMARY ---
Stage 4 raw VLM    : 0.203 (20.3%)
Stage 6 LLM corr   : 0.140 (14.0%)
Stage 7 rules      : 0.140 (14.0%)
Total improvement  : 6.3%
source_2_1744             |  13.96% |  37.04%
  GT pages found : [1]
  Page 1        : 25 lines
Running pipeline on: /kaggle/input/datasets/indrasn0wal/humanai-handwriting-ocr/handwriting/text scans/Pleito entre
el Marque_s de Viana.pdf (Page 1)
```

```
==================================================
No profile found — building from image...
  Building profile for source_3_pleito...
Output directory: /kaggle/working/outputs
  Hand profile built successfully
  Saved → source_3_pleito_profile.json
  Preprocessing validated.

Stage 2: GPT-4.1 layout masking...
  Main text bbox: x=[0.09, 0.93] y=[0.08, 0.93] conf=high
  Excluded: Bounding box includes a large decorative initial 'S' (dropcap) at the start of the main text, as well as
a left-anchored header ('GOBIERNO') that extends further left than the main text block. The bottom boundary includes
notarial marks and rubrics. Marginalia such as the folio number, outer summary, and modern pencil annotations are ex
cluded.

Stage 3: Dropcap detection...
  Dropcap: "S" (conf=high)
  First word: "SEÑOR"
  Reasoning: "Large ornate S spanning approximately 4 lines at the beginning of the main text "

Stage 4: Transcribing...
  Slicing page into 3 overlapping tiles...
  Transcribing TOP tile...
  Transcribing MID tile...
  Transcribing BOT tile...
  Merged transcription: 28 lines

Stage 6: LLM spelling correction...
  Correction using image + text...
  JSON parse failed: Invalid \escape: line 2 column 1045 (char 1046)
  Recovered from partial JSON

Stage 7: Applying deterministic rules...
  Rules applied: ç → z, q~ → que

--- FINAL TRANSCRIPTION ---
327-49
V. S.
S
N EL PLEYTO que se ha
nueuo entre el Marques de Viana
```

```
y Dona Eluira pimentel su hermana
como hija y heredera del Marques Don Pedro Pimentel
su padre difto con don Juan a espinoza pimentel Conde de Be-
avente como hijo y heredero de Don Antonio pimentel
su padre difto Conde que fue de Benavente.
Se presupone en el seño que Don Alonso pimentel Conde
de Benavente aquelo de los litigantes como de su Magd
ynfundo de dos quentos de renta en cada un año para que
los gozasse el Sr Don Antonio por los dias de su vida y
despues del Srque Marques Don Pedro su hermano por
prezio de 16. qo. e con anuadidos que el Sr Don Alonso
pago por la compra del Sr Juan y se despacho preuillegio del
por las zhas dos vidas sobre ciertas rentas en el contenido
y por las palabras del Sr preuillegio importantes para la deter-
minacion deste pleito los siguientes.
1a clausula
Se a dezir el dado preuillegio dellos al Sr Don Antonio primer
tal su fijo para que goze dellos en cada su anio parez-\zcida su vida, y que por fin y muerte del dho Don Anto-
io pimentel subceda y goze dellos don Pedro pimentel
su hermano su fijo segundo. Del dho conde de Benavente
y por muerte del dho don Pedro pimentel se consuman
todos los oficios dos quentos dentro en los nuestros
  WARNING: Line count mismatch — GT=25 lines, Pred=28 lines
  WARNING: Line count mismatch — GT=25 lines, Pred=27 lines
  WARNING: Line count mismatch — GT=25 lines, Pred=27 lines

--- CER SUMMARY ---
Stage 4 raw VLM    : 0.149 (14.9%)
Stage 6 LLM corr   : 0.146 (14.6%)
Stage 7 rules      : 0.147 (14.7%)
Total improvement  : 0.2%
source_3_pleito           |  14.72% |  39.82%
  GT pages found : [1]
  Page 1        : 21 lines
Running pipeline on: /kaggle/input/datasets/indrasn0wal/humanai-handwriting-ocr/handwriting/text scans/ES.28079.AHN_
INQUISICIO_N_1667_Exp.12 _ 1640.pdf (Page 1)
==================================================
No profile found — building from image...
  Building profile for source_4_inquisicion...
Output directory: /kaggle/working/outputs
  Hand profile built successfully
```

```
  Saved → source_4_inquisicion_profile.json
  Preprocessing validated.

Stage 2: GPT-4.1 layout masking...
  Main text bbox: x=[0.07, 0.96] y=[0.08, 0.93] conf=high
  Excluded: Bounding box includes anchoring elements at the top, the main text block, and notarial marks/signatures
at the left and bottom. No large decorative dropcaps are present, but the left boundary is expanded to include a lef
tward notarial flourish. Marginalia and page numbers are excluded.

Stage 3: Dropcap detection...
  No dropcap in this source — skipping.

Stage 4: Transcribing...
  Slicing page into 3 overlapping tiles...
  Transcribing TOP tile...
  Transcribing MID tile...
  Transcribing BOT tile...
  Merged transcription: 21 lines

Stage 6: LLM spelling correction...
  Correction using image + text...
  Corrections made: 15
  Sample corrections:
    [spelling|high] "offo" → "offo" (Matches manuscript, archaic spelling for 'año offo)
    [spelling|high] "niuonu" → "niuona" (The manuscript clearly reads 'niuona', not 'niuonu)
    [spelling|high] "Leçiana" → "Leziana" (ç is to be transcribed as z per curator rules.)
    [spelling|high] "codico" → "codico" (Matches manuscript, archaic spelling for 'código'.)
    [spelling|high] "cominos" → "cominos" (Matches manuscript, likely intended as 'comienzos')

Stage 7: Applying deterministic rules...
  Rules applied: ç → z, q~ → que

--- FINAL TRANSCRIPTION ---
En este año offo se han visto los papeles, y de la
niuona tocantes a Ana Leziana Hernandez Herna-
ni, y se a parecido que con toda breuedad vaja ala
dja villa y el primer Domingo en la yglesia della Sagra-
ver el codico de la fe que cominos con esta de tiempo
el ofentorio de la misa mayor, y el Domingo siguiente
la anathema, y si en rah dellos resultaren algunas
de laziones, las remita el comisario a primera avizo
```

```
santo offe. Y procurara que algunos sacerdotes de buena
vida (y no de otra persona aunque sea religioso) exor-
cicen alos que fina de lezanos, y de lo que resultare de los
exorcismos se hara relacion firmada de los exhor-
cistas, estando presente el comisario con adbertencia
de que estos exorcismos no se han de hacer inquisicion
de secreto, de manera que no entiendan los de aque
va loque resultare dellos, y alpazientemente sele obligara
al secreto, que para avidos los autos y lo anexo y de
pendientes se damos poder y comision baxo
ma por la presente la qual por la portareca
de los autos que sobre esto dizierpan a nomi-
no los originales conserven de anofranca noſa
  Line count match: 21 lines ✓
  Line count match: 21 lines ✓
  Line count match: 21 lines ✓

--- CER SUMMARY ---
Stage 4 raw VLM    : 0.179 (17.9%)
Stage 6 LLM corr   : 0.178 (17.8%)
Stage 7 rules      : 0.176 (17.6%)
Total improvement  : 0.3%
source_4_inquisicion      |  17.60% |  45.50%
  GT pages found : [1]
  Page 1        : 30 lines
Running pipeline on: /kaggle/input/datasets/indrasn0wal/humanai-handwriting-ocr/handwriting/text scans/AHPG-GPAH AU6
1_2 _ 1606.pdf (Page 1)
==================================================
No profile found — building from image...
  Building profile for source_5_1606...
Output directory: /kaggle/working/outputs
  Hand profile built successfully
  Saved → source_5_1606_profile.json
  Preprocessing validated.

Stage 2: GPT-4.1 layout masking...
  Main text bbox: x=[0.07, 0.97] y=[0.04, 0.97] conf=high
  Excluded: A large decorative dropcap 'E' is present at the start and included. The left boundary is expanded to in
clude the dropcap and a slightly leftward header. The bottom includes notarial marks and rubrics. No marginalia or m
odern annotations are included.
```

```
Stage 3: Dropcap detection...
  VLM confirms no dropcap on this page.

Stage 4: Transcribing...
  Slicing page into 3 overlapping tiles...
  Transcribing TOP tile...
  Transcribing MID tile...
  Transcribing BOT tile...
  Merged transcription: 30 lines

Stage 6: LLM spelling correction...
  Correction using image + text...
  Corrections made: 14
  Sample corrections:
    [missing_line|high] "" → "No missing lines detected; all visible lines in the image are present in the draft tra
nscription." (Careful comparison of the image and draft shows al)
    [spelling|high] "premiso" → "permiso" (The word 'premiso' is a garbled form of 'permiso',)
    [spelling|high] "acuse criminal mente" → "acuso criminal mente" (The correct verb form is 'acuso', matching the
sub)
    [spelling|high] "Presuetero" → "Presbitero" (The manuscript reads 'Presbitero', a common eccles)
    [spelling|high] "Recojimieto" → "Recojimiento" (The correct word is 'Recojimiento', meaning 'modes)

Stage 7: Applying deterministic rules...
  Rules applied: ç → z, q~ → que

--- FINAL TRANSCRIPTION ---
Iena de aguirre Vesina desta Villa de oñate muger legitima de maria Andres de arregui mi Hija legitima de min de arr
egui mi marido resi- dente muchos años ha enlas yndias con la Vena que a V.m. pido en falta dela marital y siendome
concedida para loque dire de yuso pa- resco ante V.m. por la mejor forma que Hubiere Lugar de derecho y premiso lo n
ecesario acuso criminal mente a santiago de garceagoitia lan- daverde clerigo Presuetero y natural de aRigorriaga en
el señorio de Visc- caya y Haziendo Verdadera relacion demi querella, es ansi que como es notorio y por tal alego qu
e Teniendo yo a da d̃a mari andres mi hija donzella en cavillo en mi casa y campania enella d̃a Villa con mucha honest
idad y Recojimieto de Heras de d̃d̃ y nueve años poco mas o menos el dho acusado siendo el susodicho matriculado enell
a Uniuer sidad y ordenado de orden sacra por lo qual el temor de Dios y Jesu Christo nunca y en gran perjuicio de mi
honra y reputacion y de la d̃a mi Hija abra rezonado por mas o menos de Taxo, de muchos requerimi entos y palabras la
reduxo a su Gusto y Voluntad y la hizo perder der su linpieza y honestidad y sacandola por fuerza y contra mi volu n
tad de las d̃as mis casas la lleuo al dho lugar de aRigorri- ga, donde la d̃a tenido siempre y atiene al presente enlo
que cometio muy atroces delitos dignos de exemplar punicion se guio e incurrió en muchas penas establecidas por dere
cho com tues de ellos reynos Por tanto pido y supp a V.m. auda Po zencia ynottoria elama Relacion declare aldho sant
iago de goi- goitia por perpetrador deles dhos delitos y le condene y exe- en su persona y bienes las dhas penas y l
as mayores que en el so lugar subiere. y ynciente mente y desu officio conzene al dho santiago aquede y pague por do
```

```
te dela dha mari andres quinien- tos ducados los quales y muchos mas tenia y espezaua tener al tpo que con el se enc
ontro desus legitimas paterna y materna y delasp- as memorias que en ella dha Villa dexaron min y banes de herad
  Line count match: 30 lines ✓
  WARNING: Line count mismatch — GT=30 lines, Pred=1 lines
  WARNING: Line count mismatch — GT=30 lines, Pred=1 lines

--- CER SUMMARY ---
Stage 4 raw VLM    : 0.110 (11.0%)
Stage 6 LLM corr   : 0.122 (12.2%)
Stage 7 rules      : 0.121 (12.1%)
Total improvement  : -1.0%
source_5_1606             |  12.06% |  46.39%

==============================================
MACRO-AVERAGE CER: 13.05%
Detailed logs saved to: /kaggle/working/test_result.json
```

In [ ]: